WEEK 1 · DAY 7 · CAPSTONE PROJECT

Week 1 Project

Model Selection Dashboard

Haiku 4.5 vs Sonnet 5 vs Opus 4.8 — Quality, Speed, Cost

This project applies all 7 days of Week 1 fundamentals: tokenization (Day 1), embeddings (Day 2), transformer architecture (Day 3), training lifecycle (Day 4), scaling laws (Day 5), structured output (Day 6). You build a working benchmark that measures three Claude models across quality, speed, and cost — and visualises the results in a decision dashboard.

6	**Benchmark Runner Code**	Full Python implementation — run all 3 models on all 4 tasks
7	**Matplotlib Dashboard**	4-panel visualisation: quality, latency, cost, trade-off
8	**FAANG Model Selection**	How Netflix, Amazon, Meta, Google choose models
9	**Architect's Lens**	Decision framework: which model for which scenario
10	**Hands-On Project**	Step-by-step: run benchmark, generate dashboard, interpret results

Date: Sunday, July 12, 2026 | Duration: 4 hours | Prerequisites: Days 1–6 | Deliverable: Working dashboard

1. The Model Selection Problem

Every AI system you build in production makes a model selection decision. Pick too small a model and quality degrades. Pick too large and costs spiral. The wrong choice is not a technical mistake — it is a business mistake.

The Stakes at Scale

At 10 million API calls/day: using Opus 4.8 instead of Haiku 4.5 where Haiku is sufficient costs $140,000 more per day. That is $51M/year of wasted spend. Model selection is not an afterthought — it is a core architectural decision.

Three Variables — Always in Tension

Quality	Speed	Cost
Accuracy of answers, reasoning depth, instruction following, edge case handling	Time to first token, total latency, tokens/second, P95 latency under load	$/1M input tokens, $/1M output tokens, total cost at production volume
Opus 4.8 wins	**Haiku 4.5 wins**	**Haiku 4.5 wins**

Why You Need a Dashboard — Not a Gut Feeling

Most teams pick a model once and never revisit it. They choose Sonnet because 'it seems good' — without measuring whether Haiku achieves 95% of the quality at 25% of the cost for their specific task. The Model Selection Dashboard turns a gut-feel decision into a data-driven one.

2. The Three Claude Models

Three models, three positions on the quality-speed-cost triangle. Understanding where each sits — and why — is fundamental to model selection.

Model Overview

Model	ID	Position	Context Window	Best For
Claude Haiku 4.5	claude-haiku-4-5-20251001	Fast + Cheap	200K tokens	High-volume, latency-sensitive tasks
Claude Sonnet 5	claude-sonnet-5	Balanced	200K tokens	Production default — quality + reasonable cost
Claude Opus 4.8	claude-opus-4-8	Highest quality	200K tokens	Complex reasoning, highest-stakes decisions

Pricing — The Numbers You Must Know

Pricing is per million tokens. Every model has separate input and output rates because generating tokens (output) requires more compute than processing them (input). Output tokens are always 3–5x more expensive than input tokens.

Model	Input ($/1M tokens)	Output ($/1M tokens)	Cost ratio vs Haiku
Haiku 4.5	$0.80	$4.00	1x (baseline)
Sonnet 5	$3.00	$15.00	3.75x input / 3.75x output
Opus 4.8	$15.00	$75.00	18.75x input / 18.75x output

At-Scale Cost Reality

COST AT SCALE — 10M calls/day

```
# 10 million calls/day, average 500 input + 200 output tokens per call

daily_input_tokens  = 10_000_000 * 500   # = 5B tokens
daily_output_tokens = 10_000_000 * 200   # = 2B tokens

haiku_daily  = (5_000 * 0.80) + (2_000 * 4.00)    # $4,000 + $8,000   = $12,000/day
sonnet_daily = (5_000 * 3.00) + (2_000 * 15.00)   # $15,000 + $30,000 = $45,000/day
opus_daily   = (5_000 * 15.00) + (2_000 * 75.00)  # $75,000 + $150,000= $225,000/day

print(f'Haiku 4.5:  ${haiku_daily:,.0f}/day  = ${haiku_daily*365/1e6:.1f}M/year')
# Haiku 4.5:  $12,000/day  = $4.4M/year
# Sonnet 5:   $45,000/day  = $16.4M/year
# Opus 4.8:   $225,000/day = $82.1M/year
```

```
# Switching Opus → Haiku where quality is sufficient: $77.7M/year saved
```

3. Benchmark Design — 4 Task Types

A benchmark is only useful if it reflects your actual production tasks. We use 4 task types that cover the full spectrum of LLM applications — from simple classification to multi-step reasoning.

Task 1 — Reasoning	
Definition	Multi-step logical or mathematical problem requiring chain-of-thought.
Examples	Complex customer support routing, financial analysis, diagnostic reasoning
Metrics	Steps taken, correctness, confidence score
Verdict	**Opus wins clearly — reasoning depth scales with model size**

Task 2 — Extraction	
Definition	Pull specific structured fields from unstructured text (names, dates, amounts).
Examples	Invoice processing, contract review, news entity extraction
Metrics	Field accuracy, completeness, hallucination rate
Verdict	**Haiku often sufficient — extraction is pattern-matching, not reasoning**

Task 3 — Summarisation	
Definition	Compress a long document into key points while preserving meaning.
Examples	Earnings call summaries, support ticket triage, news briefings
Metrics	Coverage, conciseness, factual accuracy
Verdict	**Sonnet sweet spot — needs language quality but not deep reasoning**

Task 4 — Classification	
Definition	Assign one of N predefined categories to a piece of text.
Examples	Sentiment analysis, content moderation, intent detection
Metrics	Accuracy on known labels, speed, consistency across runs
Verdict	**Haiku wins on cost — classification rarely needs large models**

4. Structured Metric Collection (Day 6 Applied)

Every benchmark result is captured as a schema-enforced Pydantic model using the tool use pattern from Day 6. This gives you typed, queryable benchmark data with zero parsing fragility.

BENCHMARK RESULT SCHEMA — Pydantic model

```
from pydantic import BaseModel, Field
from typing import Literal, List, Optional
import time

# ■■ Benchmark result schema ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class BenchmarkResult(BaseModel):
    model: str = Field(description='Model ID used for this call')
    task_type: Literal['reasoning','extraction','summarisation','classification']
    task_id: str = Field(description='Unique task identifier')

    # Quality metrics
    answer: str = Field(description='The model answer or extracted content')
    reasoning_steps: int = Field(ge=0,
        description='Number of distinct reasoning steps taken (0 if none)')
    confidence: Literal['high','medium','low'] = Field(
        description='Model confidence in its answer')

    # Performance metrics
    latency_ms: int = Field(ge=0, description='Total API call latency in ms')
    input_tokens: int = Field(ge=0)
    output_tokens: int = Field(ge=0)

    # Cost (computed from tokens)
    cost_usd: float = Field(ge=0.0, description='Cost of this single call in USD')

    # Errors
    error: Optional[str] = Field(default=None,
        description='Error message if call failed, else null')
```

Token Pricing Constants

PRICING CONSTANTS + COST FUNCTION

```
# Pricing per 1 million tokens (as of July 2026)
PRICING = {
    "claude-haiku-4-5-20251001": {"input": 0.80,  "output": 4.00},
    "claude-sonnet-5":           {"input": 3.00,  "output": 15.00},
    "claude-opus-4-8":           {"input": 15.00, "output": 75.00},
}

def compute_cost(model: str, input_tokens: int, output_tokens: int) -> float:
    p = PRICING[model]
    return (input_tokens * p['input'] + output_tokens * p['output']) / 1_000_000

# Example:
cost = compute_cost('claude-sonnet-5', 500, 200)
print(f'${cost:.6f}')  # $0.004500 per call
```

Single Benchmark Call Function

RUN_SINGLE — one model × one task → BenchmarkResult

```
import anthropic

client = anthropic.Anthropic()

RESULT_TOOL = {
    "name": "return_result",
    "description": "Return the task result and metadata",
    "input_schema": BenchmarkResult.model_json_schema()
}

def run_single(model: str, task: dict) -> BenchmarkResult:
    start = time.time()
    try:
        resp = client.messages.create(
            model=model,
            max_tokens=1024,
            tools=[RESULT_TOOL],
            tool_choice={"type": "tool", "name": "return_result"},
            messages=[{
                "role": "user",
                "content": task["prompt"]
            }]
        )
        elapsed_ms = int((time.time() - start) * 1000)
        raw = resp.content[0].input
        raw['model']      = model
        raw['task_type']  = task['type']
        raw['task_id']    = task['id']
        raw['latency_ms'] = elapsed_ms
        raw['input_tokens']  = resp.usage.input_tokens
        raw['output_tokens'] = resp.usage.output_tokens
        raw['cost_usd'] = compute_cost(
            model, resp.usage.input_tokens, resp.usage.output_tokens)
        return BenchmarkResult(**raw)
    except Exception as e:
        return BenchmarkResult(model=model, task_type=task['type'],
            task_id=task['id'], answer='ERROR', reasoning_steps=0,
            confidence='low', latency_ms=int((time.time()-start)*1000),
            input_tokens=0, output_tokens=0, cost_usd=0.0, error=str(e))
```

5. Cost Mathematics

Before running any benchmark, work out your cost envelope. This tells you how much quality delta justifies the price premium.

Break-Even Analysis

The question is not 'which model is cheapest?' It is: 'Is the quality improvement worth the price premium?'

BREAK-EVEN ANALYSIS — is the premium worth it?

```
# Scenario: Customer support routing — 1M tickets/day
# Each ticket: ~300 input tokens, ~100 output tokens

tickets_per_day = 1_000_000
input_tokens    = 300
output_tokens   = 100

def daily_cost(model_key):
    p = PRICING[model_key]
    return tickets_per_day * (
        input_tokens  * p['input']  / 1_000_000 +
        output_tokens * p['output'] / 1_000_000
    )

haiku_daily  = daily_cost('claude-haiku-4-5-20251001')  # $560/day
sonnet_daily = daily_cost('claude-sonnet-5')            # $2,400/day
opus_daily   = daily_cost('claude-opus-4-8')            # $12,000/day

# Sonnet premium over Haiku: $2,400 - $560 = $1,840/day = $671,600/year
# Question: does Sonnet route 0.18% more tickets correctly?
# If each mis-routed ticket costs $10 in rework: need 184 fewer errors/day to break even
# That is 0.018% accuracy improvement on 1M tickets
# Answer: Almost certainly yes — but benchmark to confirm.
```

The Right Question

Never ask 'which model is best?' Always ask: 'What is the minimum quality threshold for my task, and which model meets it most cheaply?' The benchmark gives you the data to answer this question objectively.

Quality Score Normalisation

To compare quality across tasks, normalise each task to a 0–100 score. Then compute a weighted composite score based on your task distribution.

COMPOSITE QUALITY SCORE — weighted across task distribution

```
# Example benchmark results (normalised scores 0-100):

scores = {
    'claude-haiku-4-5-20251001': {
        'reasoning':      62,   # Struggles with multi-step
        'extraction':     91,   # Excellent pattern recognition
        'summarisation':  78,   # Good but loses nuance
        'classification': 94,   # Near-perfect on known categories
```

```
    },
    'claude-sonnet-5': {
        'reasoning':      88,
        'extraction':     95,
        'summarisation':  92,
        'classification': 97,
    },
    'claude-opus-4-8': {
        'reasoning':      96,
        'extraction':     97,
        'summarisation':  96,
        'classification': 98,
    },
}
# Your task distribution (weights sum to 1.0):
weights = {'reasoning': 0.15, 'extraction': 0.40,
           'summarisation': 0.25, 'classification': 0.20}
for model, task_scores in scores.items():
    composite = sum(task_scores[t] * weights[t] for t in weights)
    print(f'{model}: {composite:.1f}/100')
# haiku:  84.7/100   sonnet: 93.7/100   opus: 96.7/100
# For this task mix, sonnet is 93.7% quality at 3.75x haiku's cost
# Opus is only 3% better than sonnet at 5x sonnet's cost
```

6. Complete Benchmark Runner

The full benchmark runner — define tasks, run all models, collect results, save to JSON. This is your complete benchmark.py file.

BENCHMARK.PY — PART 1: imports, models, pricing, tasks

```
#!/usr/bin/env python3
"""benchmark.py — Model Selection Dashboard benchmark runner"""
import anthropic, json, time
from pydantic import BaseModel, Field
from typing import Literal, Optional
client = anthropic.Anthropic()
MODELS = [
    "claude-haiku-4-5-20251001",
    "claude-sonnet-5",
    "claude-opus-4-8",
]
PRICING = {
    "claude-haiku-4-5-20251001": {"input": 0.80,  "output": 4.00},
    "claude-sonnet-5":           {"input": 3.00,  "output": 15.00},
    "claude-opus-4-8":           {"input": 15.00, "output": 75.00},
}
# ■■ Task Suite ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
TASKS = [
    {
        "id": "reasoning_01",
        "type": "reasoning",
        "prompt": """A train leaves City A at 08:00 travelling at 120 km/h.
A second train leaves City B (480 km away) at 09:30 travelling toward City A
at 90 km/h. At what time do they meet, and how far from City A?
Show your full working step by step."""
    },
    {
        "id": "extraction_01",
        "type": "extraction",
        "prompt": """Extract entities from this text.
Netflix Q4 2025 earnings: Revenue $10.2B (up 16% YoY),
paid memberships 301M, EPS $4.27.
CEO Ted Sarandos cited APAC strength. HQ: Los Gatos CA.
CFO: Spencer Neumann.
Return: company, quarter, revenue_usd, growth_pct,
members_M, eps, ceo, cfo, hq_city"""
    },
    {
        "id": "summarisation_01",
        "type": "summarisation",
        "prompt": """Summarise this in exactly 3 bullet points, max 20 words each:
Amazon Web Services reported cloud revenue of $28.8B in Q4 2025, growing 19% YoY.
The growth was driven by AI workloads including model training and inference.
```

```
Microsoft Azure grew 33% in the same period. Google Cloud grew 28%.
AWS maintains 31% market share vs Azure 21% and Google Cloud 12%.
Key customers include Netflix, Meta, and Goldman Sachs for AI workloads."""
    },
    {
        "id": "classification_01",
        "type": "classification",
        "prompt": """Classify this customer message into exactly one category:
Categories: BILLING_ISSUE, TECHNICAL_SUPPORT, ACCOUNT_ACCESS, FEATURE_REQUEST, COMPLAINT
Message: 'I have been trying to log in for 3 days and keep getting error code 403.
I reset my password twice and it still does not work. This is extremely frustrating
and I have an important presentation tomorrow.'"""
    },
]
```

BENCHMARK.PY — PART 2: schema, runner, main

```
class BenchmarkResult(BaseModel):
    model: str
    task_type: Literal['reasoning','extraction','summarisation','classification']
    task_id: str
    answer: str
    reasoning_steps: int = Field(ge=0)
    confidence: Literal['high','medium','low']
    latency_ms: int
    input_tokens: int
    output_tokens: int
    cost_usd: float
    error: Optional[str] = None
RESULT_TOOL = {
    "name": "return_result",
    "description": "Return the task answer and performance metadata",
    "input_schema": BenchmarkResult.model_json_schema()
}
def run_single(model: str, task: dict) -> BenchmarkResult:
    start = time.time()
    try:
        resp = client.messages.create(
            model=model, max_tokens=1024,
            tools=[RESULT_TOOL],
            tool_choice={"type": "tool", "name": "return_result"},
            messages=[{"role": "user", "content": task["prompt"]}]
        )
        elapsed = int((time.time() - start) * 1000)
        raw = resp.content[0].input
        raw.update({
            "model": model, "task_type": task["type"],
            "task_id": task["id"], "latency_ms": elapsed,
            "input_tokens":  resp.usage.input_tokens,
            "output_tokens": resp.usage.output_tokens,
            "cost_usd": (resp.usage.input_tokens  * PRICING[model]["input"] +
```

```
                        resp.usage.output_tokens * PRICING[model]["output"]
                       ) / 1_000_000
        })
        return BenchmarkResult(**raw)
    except Exception as e:
        return BenchmarkResult(
            model=model, task_type=task['type'], task_id=task['id'],
            answer='ERROR', reasoning_steps=0, confidence='low',
            latency_ms=int((time.time()-start)*1000),
            input_tokens=0, output_tokens=0, cost_usd=0.0, error=str(e))

def run_all() -> list[BenchmarkResult]:
    results = []
    for model in MODELS:
        for task in TASKS:
            print(f'Running {model} / {task["id"]}...')
            r = run_single(model, task)
            results.append(r)
            time.sleep(0.5)  # rate limit buffer
    return results

if __name__ == '__main__':
    results = run_all()
    data = [r.model_dump() for r in results]
    with open('benchmark_results.json', 'w') as f:
        json.dump(data, f, indent=2)
    print(f'Done. {len(data)} results saved to benchmark_results.json')
```

7. Matplotlib Dashboard

The dashboard reads benchmark_results.json and renders a 4-panel figure: quality by task, latency comparison, cost per task, and the quality-vs-cost scatter that drives the final model recommendation.

VISUALIZE.PY — PART 1: load + index

```
#!/usr/bin/env python3
"""visualize.py — Model Selection Dashboard"""
import json, numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
# ■■ Load results ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
with open('benchmark_results.json') as f:
    raw = json.load(f)
MODELS = [
    "claude-haiku-4-5-20251001",
    "claude-sonnet-5",
    "claude-opus-4-8",
]
MODEL_LABELS = ["Haiku 4.5", "Sonnet 5", "Opus 4.8"]
TASKS = ["reasoning", "extraction", "summarisation", "classification"]
COLORS = ["#0d5c63", "#6b21a8", "#c2410c"]  # teal, purple, orange
# Index results by model + task
def get(model, task, field):
    matches = [r for r in raw if r['model']==model and r['task_type']==task]
    return matches[0][field] if matches else 0
# Quality: map confidence to score (placeholder for real eval)
CONF_SCORE = {'high': 95, 'medium': 75, 'low': 45}
quality = {
    m: [CONF_SCORE[get(m, t, 'confidence')] for t in TASKS]
    for m in MODELS
}
latency  = {m: [get(m, t, 'latency_ms')  for t in TASKS] for m in MODELS}
cost_per = {m: [get(m, t, 'cost_usd') * 1000 for t in TASKS] for m in MODELS}
```

VISUALIZE.PY — PART 2: 4-panel Matplotlib dashboard

```
# ■■ Build 4-panel dashboard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Model Selection Dashboard — Week 1 Project',
             fontsize=16, fontweight='bold', y=0.98)
x = np.arange(len(TASKS))
width = 0.25
# Panel 1: Quality by task
ax1 = axes[0, 0]
for i, (model, label) in enumerate(zip(MODELS, MODEL_LABELS)):
    ax1.bar(x + i*width, quality[model], width, label=label,
            color=COLORS[i], alpha=0.85)
ax1.set_title('Quality Score by Task', fontweight='bold')
```

```
ax1.set_xticks(x + width); ax1.set_xticklabels(TASKS, rotation=15)
ax1.set_ylim(0, 105); ax1.set_ylabel('Score (0–100)')
ax1.axhline(y=90, color='grey', linestyle='--', alpha=0.5, label='Target (90)')
ax1.legend(); ax1.grid(axis='y', alpha=0.3)

# Panel 2: Latency by task
ax2 = axes[0, 1]
for i, (model, label) in enumerate(zip(MODELS, MODEL_LABELS)):
    ax2.bar(x + i*width, latency[model], width, label=label,
            color=COLORS[i], alpha=0.85)
ax2.set_title('Latency by Task (ms)', fontweight='bold')
ax2.set_xticks(x + width); ax2.set_xticklabels(TASKS, rotation=15)
ax2.set_ylabel('Latency (ms)'); ax2.legend(); ax2.grid(axis='y', alpha=0.3)

# Panel 3: Cost per 1000 calls
avg_cost = {m: np.mean(cost_per[m]) for m in MODELS}
ax3 = axes[1, 0]
bars = ax3.bar(MODEL_LABELS,
               [avg_cost[m] for m in MODELS],
               color=COLORS, alpha=0.85, width=0.5)
ax3.set_title('Average Cost per 1,000 Calls (USD)', fontweight='bold')
ax3.set_ylabel('Cost (USD)')
for bar, model in zip(bars, MODELS):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'${avg_cost[model]:.3f}', ha='center', va='bottom', fontsize=9)
ax3.grid(axis='y', alpha=0.3)

# Panel 4: Quality vs Cost scatter (the decision plot)
ax4 = axes[1, 1]
avg_quality = {m: np.mean(quality[m]) for m in MODELS}
for i, (model, label) in enumerate(zip(MODELS, MODEL_LABELS)):
    ax4.scatter(avg_cost[model], avg_quality[model],
                s=200, color=COLORS[i], zorder=5)
    ax4.annotate(label,
                 (avg_cost[model], avg_quality[model]),
                 textcoords='offset points', xytext=(8, 4), fontsize=10)
ax4.set_title('Quality vs Cost Trade-off', fontweight='bold')
ax4.set_xlabel('Avg Cost per 1,000 Calls (USD)')
ax4.set_ylabel('Avg Quality Score (0–100)')
ax4.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('model_selection_dashboard.png', dpi=150, bbox_inches='tight')
plt.show()
print('Dashboard saved to model_selection_dashboard.png')
```

8. FAANG Model Selection Decisions

Real model selection decisions from major AI products — each one driven by the same quality-speed-cost framework you are building.

Netflix — Three-Tier Model Architecture	
Volume: 260M subscribers	100M+ LLM calls/day
Strategy	Three-tier: Haiku for real-time (recommendations, search ranking, subtitle quality flags). Sonnet for async (content tagging, personalised email subject lines, A/B test analysis). Opus for weekly batch (long-form content strategy, market analysis reports).
Key insight	Never use a high-cost model for a task that runs 50M times/day. Haiku recommendation engine: $4,000/day. Sonnet equivalent: $45,000/day. Opus equivalent: $225,000/day. Netflix saves $221K/day using Haiku for recommendations alone.

Amazon — Product Attribute Extraction	
Volume: 500M+ product listings	Updated continuously
Strategy	Haiku for attribute extraction (category, brand, size, colour, material). Sonnet for product descriptions (requires language quality + creativity). Opus never used in automated pipelines — reserved for human-assisted edge cases and category taxonomy creation.
Key insight	Extraction accuracy at 91% (Haiku) vs 95% (Sonnet) — a 4% quality gap is worth $41M/year in cost savings at Amazon's scale. They accept the gap and handle edge cases in a separate Sonnet queue.

Meta — Content Moderation Tiering	
Volume: 3B+ posts/day	Real-time decisions required
Strategy	Haiku for first-pass screening (safe/review/remove in <200ms). Sonnet for 'review' decisions from first pass — policy ambiguity resolution. Opus for appeals, novel violation categories, and policy team research. 95%+ of traffic never touches Sonnet.
Key insight	Tier 1 (Haiku) catches 90%+ of violations cheaply. Only ambiguous cases escalate. This 3-tier architecture reduces per-violation cost by 85% vs a flat Sonnet-for-all approach.

Google — Search Quality Assessment	
Volume: Millions of rating tasks/month	Training data generation
Strategy	Sonnet for most quality rating tasks (relevance, helpfulness, accuracy). Opus for novel evaluation dimensions, new benchmark creation, and cross-checking Sonnet ratings for calibration.
Key insight	Opus as calibration for Sonnet — run 5% of tasks through Opus to measure Sonnet's systematic biases. Correct Sonnet ratings statistically rather than switching to Opus for all tasks.

9. Architect's Lens — Model Selection Framework

Four questions that determine model selection for any production system. Answer them in order — the first 'no' is where you stop.

Question 1: Does the task require multi-step reasoning?

Reasoning tasks (planning, diagnosis, complex analysis) scale with model size. If yes and quality matters, start with Sonnet. Benchmark Opus if Sonnet falls short. If no — go to Question 2.

Examples: Routing customer tickets: NO (classification). Writing a financial report: YES.

Question 2: Is latency under 500ms required?

Real-time UX (search results, autocomplete, recommendations) needs sub-500ms. Haiku averages 200–400ms. Sonnet averages 600ms–1.5s. Opus is 2–5s. If sub-500ms is hard: Haiku only.

Examples: Netflix play button recommendation: YES → Haiku only. Nightly batch report generation: NO → all models viable.

Question 3: What is the volume (calls/day)?

Below 100K/day: cost is irrelevant — use the best model you can afford. 100K–10M/day: Sonnet vs Haiku breakeven analysis required. Above 10M/day: Haiku unless you have clear business case for Sonnet quality delta.

Examples: Internal analytics tool (5K calls/day): use Sonnet without analysis. Public API (50M calls/day): must benchmark — every 1% quality difference costs or saves $100K+/year.

Question 4: What is the cost of an error?

Low-stakes tasks (product category labels, search snippets): accept Haiku's lower accuracy — the volume savings outweigh the error rate. High-stakes tasks (legal document review, medical information, financial advice): quality trumps cost — Opus is the minimum viable model.

Examples: Amazon product title: LOW stakes → Haiku. Contract clause extraction for legal team: HIGH stakes → Opus + human review.

Quick Reference Decision Table

Scenario	Recommended Model	Reason
Real-time user-facing (<500ms)	Haiku 4.5	Latency constraint
High-volume simple tasks (>1M/day)	Haiku 4.5	Cost — quality sufficient

Production default (balanced)	Sonnet 5	Best quality-cost for most tasks
Complex reasoning / planning	Sonnet 5 or Opus 4.8	Benchmark to decide
Legal / medical / financial (high stakes)	Opus 4.8	Error cost too high
Nightly batch analytics (<10K/day)	Sonnet 5	Volume low — use quality
A/B testing model comparison	All three + benchmark	Data-driven decision

10. Hands-On Project — Step by Step

Build the complete Model Selection Dashboard. Estimated time: 3–4 hours. Save all files to ~/Desktop/AI-ML/week1-project/

Setup

```
mkdir -p ~/Desktop/AI-ML/week1-project

cd ~/Desktop/AI-ML/week1-project

pip install anthropic pydantic matplotlib numpy

export ANTHROPIC_API_KEY=your_key_here
```

Step 1 — Create benchmark.py (30 min)

- Copy the complete benchmark.py from Sections 4 and 6
- Add your ANTHROPIC_API_KEY to the environment
- Run: python3 benchmark.py
- Expected output: 12 lines (3 models × 4 tasks) — takes ~2 minutes
- Verify: benchmark_results.json exists with 12 entries

Step 2 — Inspect Raw Results (20 min)

- Open benchmark_results.json and read every result
- For each result, note: answer quality, confidence, latency_ms, cost_usd
- Manually score each answer 0–100 and add a 'manual_score' field
- This manual scoring is your ground truth for the quality axis

Step 3 — Create visualize.py (45 min)

- Copy visualize.py from Section 7
- Replace CONF_SCORE logic with your manual_score values from Step 2
- Run: python3 visualize.py
- Open model_selection_dashboard.png
- You should see: 4 panels showing quality, latency, cost, trade-off scatter

Step 4 — Analyse Your Dashboard (30 min)

- Look at Panel 4 (Quality vs Cost scatter) — where does each model sit?
- For each task type: which model hits 90%+ quality at lowest cost?

- Calculate: what is the annual cost saving if you used the cheapest model that hits 90% quality for each task?
- Write 3 bullet points: your model recommendation for each scenario

Step 5 — Extend With Your Own Tasks (60 min)

- Add 2 tasks from your own production domain (data querying, SQL generation, report drafting)
- Add them to the TASKS list in benchmark.py
- Re-run the benchmark and regenerate the dashboard
- This makes the benchmark relevant to your actual use case

Expected Findings (Typical Results)

Haiku 4.5: classification ~94%, extraction ~91%, summarisation ~78%, reasoning ~62% — fast (<400ms), cheapest

Sonnet 5: all tasks >88%, best balance — moderate latency (~800ms), 3.75x Haiku cost

Opus 4.8: all tasks >95%, best reasoning — slowest (2–4s), 18.75x Haiku cost

Key insight: For extraction + classification (likely 60%+ of your workload), Haiku matches Sonnet quality at 3.75x lower cost

FINAL PROJECT STRUCTURE

```
# Final project structure:
~/Desktop/AI-ML/week1-project/
■■■ benchmark.py             # Benchmark runner
■■■ visualize.py             # Dashboard generator
■■■ benchmark_results.json   # Raw results (12 entries)
■■■ model_selection_dashboard.png  # Your deliverable
# Push to GitHub:
cp -r ~/Desktop/AI-ML/week1-project/ ~/AI-Engineering/week-01/project/
cd ~/AI-Engineering
git add week-01/project/
git commit -m "Week 1 project: Model Selection Dashboard"
git push origin main
```

Week 1 Complete — Summary & Week 2 Preview

Week 1 — What You Have Built

Day 1 — Tokenization	BPE, WordPiece, Unigram, tiktoken, vocab sizes 50K–256K
Day 2 — Embeddings	Word2Vec→BERT evolution, SBERT, bi/cross-encoder, MTEB, Voyage
Day 3 — Transformer Architecture	Attention, RoPE, ALiBi, FlashAttention, KV Cache, 175B GPT-3
Day 4 — Training Lifecycle	Pre-train→SFT→RLHF/DPO/CAI, C=6ND, Chinchilla D=20N
Day 5 — Scaling Laws + Prompts	Kaplan/Chinchilla laws, emergence, CoT, injection defence
Day 6 — Structured Output	JSON Schema, constrained decoding, Pydantic, Text-to-SQL Piece 1
Day 7 — Model Selection	Benchmark design, Haiku/Sonnet/Opus, cost mathematics, dashboard

Text-to-SQL Capstone — Piece 1 Complete

Piece	What It Does	Status
1. Structured Output (Week 1)	SQL as schema-enforced JSON, confidence + validation	**DONE**
2. Schema RAG (Week 3)	Dynamic table retrieval for 500+ tables	**Upcoming**
3. Self-Correcting Agent (Week 5)	Execute SQL → error → retry loop	**Upcoming**
4. Evaluation Harness (Week 7)	Execution accuracy on 50 benchmark questions	**Upcoming**
5. Multi-Agent Architecture (Week 8)	Planner + Generator + Validator + Executor	**Upcoming**
6. Security Layer (Week 10)	Prompt injection → SQL injection defence	**Upcoming**
7. LLMOps Dashboard (Week 11)	Cost, latency, accuracy monitoring	**Upcoming**
8. Full Capstone (Week 12)	Complete production system	**Upcoming**

Week 2 Preview — Quantization, Fine-Tuning & Serving

Monday (Day 8): Quantization — INT8, INT4, GPTQ, AWQ, GGUF. How to run 70B models on a single GPU.

Tuesday (Day 9): LoRA + PEFT — fine-tune without full retraining. Adapter layers, rank, alpha, merging.

Wednesday (Day 10): Full Fine-Tuning — when to fine-tune vs RAG vs prompting.

Thursday (Day 11): Model Serving — vLLM, TGI, continuous batching, PagedAttention, throughput optimisation.

Friday (Day 12): Evaluation — MMLU, HumanEval, MT-Bench, LLM-as-judge.

Saturday (Day 13): Efficiency deep dive — KV Cache tuning, speculative decoding.

Sunday (Day 14): Week 2 Project — Fine-tune a model on your domain data.

EAGLE EYE ADDENDUM — DAY 7: Open-Source vs Proprietary — The Missing Dimension

Day 7 covered model selection between Claude tiers (Haiku, Sonnet, Opus). A critical decision dimension was missing: open-source vs proprietary models — the first gate in any real-world model selection process.

The First Decision: OSS vs Proprietary

Before choosing between Claude Haiku and Claude Sonnet, architects must first decide: proprietary API (Claude, GPT-4, Gemini) or open-source self-hosted (LLaMA 3, Mistral, Qwen). This is not a technical decision — it is a business and compliance decision.

Dimension	Proprietary (Claude / GPT-4)	Open-Source (LLaMA 3 / Mistral)
Cost model	Pay per token; no infra cost	Infra cost; no per-token cost
Quality ceiling	Higher (frontier models)	Approaching parity at 70B
Data privacy	Data sent to third party	All data stays on-premises
Compliance (HIPAA/GDPR)	Requires BAA / DPA	Full control — easier compliance
Latency control	Shared infra; SLA varies	Dedicated infra; you control
Fine-tuning	Limited (GPT-4 no; Claude no)	Full fine-tuning supported
Custom dialect	Not possible	Fine-tune on your dialect
Offline / air-gap	Impossible	Possible (GGUF + llama.cpp)
Time to production	Minutes (API key)	Days-weeks (infra setup)

The Decision Framework

```
GATE 1: Is data privacy / compliance a hard requirement?
  - HIPAA (healthcare), GDPR with strict data residency,
    financial PII, government classified data
  YES -> Open-source self-hosted ONLY (LLaMA 3 70B or Mistral)
  NO  -> Both options viable; continue to Gate 2

GATE 2: Is quality ceiling the top priority?
  - Complex reasoning, frontier-level tasks
  - Need latest capabilities (vision, long context, tool use)
  YES -> Proprietary (Claude Sonnet 5 / Opus 4.8) while OSS catches up
  NO  -> Continue to Gate 3
```

```
GATE 3: What is the volume and budget?
  - < 1M calls/day:   Proprietary API cheaper (no GPU infra)
  - > 10M calls/day:  OSS self-hosted cheaper (GPU amortizes)
  - Self-host crossover: ~2-5M calls/day for 7B model
  YES (high volume) -> Open-source self-hosted
  NO  (moderate)    -> Proprietary API

GATE 4: Do you need fine-tuning or custom behavior?
  YES -> Open-source REQUIRED (Days 8-10 apply)
  NO  -> Proprietary prompt engineering often sufficient
```

The OSS Model Landscape (July 2026)

Model	Size	Quality vs GPT-4	License	Best Use Case
LLaMA 3.1 8B	8B	~70-75%	Meta LLaMA 3	High-volume, low-cost tasks
LLaMA 3.1 70B	70B	~90-95%	Meta LLaMA 3	Complex tasks, near-frontier
Mistral 7B	7B	~70%	Apache 2.0 (fully)	Commercial, code tasks
Mixtral 8x7B	47B	~85%	Apache 2.0 (fully)	MOE; cost-efficient quality
Qwen 2.5 72B	72B	~92%	Qwen License	Multilingual; Asian languages
Gemma 2 27B	27B	~83%	Gemma License	Google infra users; efficient

Cost Crossover Analysis

```
# At what volume does self-hosting LLaMA 3 70B become cheaper than Claude Haiku?
# GPU cost: 8x A100 80GB = $32/hr = ~$768/day = $280K/year
# Claude Haiku: $0.80/M input + $4.00/M output
# Assume 500 input + 200 output tokens per call:

haiku_cost_per_call = (500 * 0.80 + 200 * 4.00) / 1_000_000  # = $0.0012
gpu_daily_cost = 32 * 24  # = $768/day (8x A100 running 24/7)
llama_throughput = 8_000_000  # tokens/second for 70B on 8xA100

# Daily capacity: ~1.5M calls/day (8M tokens/sec / 700 tokens/call)
# Crossover: $768 / $0.0012 = 640,000 calls/day

# CONCLUSION:
# < 640K calls/day: Claude Haiku is cheaper
# > 640K calls/day: LLaMA 3 70B self-hosted is cheaper
# At 2M calls/day: Haiku = $2,400/day vs LLaMA = $768/day (3x savings)
```

FAANG Model Strategy

- Meta: 100% open-source (LLaMA models) — controls full stack, no per-token cost
- Google: proprietary (Gemini) for external products, open-source (Gemma) for self-serve

- Amazon: Titan (proprietary) for Bedrock + LLaMA 3 on SageMaker for fine-tuning customers
- Netflix: Claude for complex reasoning tasks + LLaMA 3 70B for high-volume tagging
- Microsoft: GPT-4 (Azure OpenAI) for external + Phi-3 / LLaMA for on-premises customers

ARCHITECT RULE: The Day 7 dashboard (Haiku vs Sonnet vs Opus) answers 'which Claude tier?'. The OSS vs proprietary framework answers the prior question: 'should we use Claude at all?' Always run Gate 1 (compliance) and Gate 3 (volume) first. Many production systems use BOTH: proprietary for quality-critical paths, OSS for high-volume commodity tasks.